

LAPORAN PRAKTIKUM STRUKTUR DATA

“PRATIKUM 6”

Disusun Oleh:

Aryahiyahul Fikra

2511532026

Dosen Pengampu:

Dr. Wahyudi S.T,M.T

Asisten Pembimbing:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Daftar Isi

Daftar Isi	1
BAB I.....	3
PENDAHULUAN	3
1.1 Latar Belakang.....	3
1.2 Tujuan Pratikum	3
BAB II	4
PEMBAHASAN.....	4
2.2 Struktur Node	4
2.3 Operasi Insert Pada DLL	5
2.4 Operasi Delete Pada DLL	7
2.5 Operasi Traversal Pada DLL	9
BAB III	10
PENUTUP	10
3.1 Kesimpulan.....	10
TUGAS DLL	11
Program Lagu_2511532026	11
Program Musik_2511532026	12
Output	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan cara untuk menyimpan dan mengatur data agar dapat digunakan secara efisien. Salah satu struktur data yang sering digunakan adalah Linked List. Pada Linked List, data disimpan dalam node yang saling terhubung menggunakan pointer.

Doubly Linked List (DLL) adalah pengembangan dari Singly Linked List yang memiliki dua pointer, yaitu pointer ke node berikutnya (next) dan pointer ke node sebelumnya (prev). Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan dari depan maupun belakang.

Pada praktikum ini dibuat beberapa program Java yang berkaitan dengan Doubly Linked List, yaitu program pembentukan node, penambahan node, penghapusan node, dan penelusuran node.

Program-program tersebut bertujuan untuk memahami cara kerja struktur data Doubly Linked List dalam operasi dasar.

1.2 Tujuan Pratikum

Tujuan pratikum ini Adalah:

1. Memahami konsep Doubly Linked List.
2. Mengetahui cara implementasi Doubly Linked List menggunakan bahasa Java.
3. Memahami operasi penambahan, penghapusan, dan penelusuran node pada Doubly Linked List.

BAB II PEMBAHASAN

2.1 Doubly Linked List

Doubly Linked List adalah struktur data linear yang terdiri dari kumpulan node yang saling terhubung dua arah. Setiap node memiliki tiga bagian, yaitu:

1. Data
2. Pointer ke node berikutnya (next)
3. Pointer ke node sebelumnya (prev)

Keunggulan Doubly Linked List dibanding Singly Linked List adalah penelusuran dapat dilakukan dari depan maupun belakang.

2.2 Struktur Node

Pada program NodeDLL_2511532026, dibuat class node yang berfungsi menyimpan data dan pointer.

Kode Program:

```
1 package pekan6_2511532026;
2
3 public class NodeDLL_2511532026 {
4     int data_2026;
5     NodeDLL_2511532026 next_2026;
6     NodeDLL_2511532026 prev_2026;
7
8     public NodeDLL_2511532026 (int data) {
9         this.data_2026 = data;
10        this.next_2026 = null;
11        this.prev_2026 = null;
12    }
13 }
14
```

Atribut yang digunakan:

1. data_2026 → menyimpan nilai data
2. next_2026 → menunjuk node berikutnya
3. prev_2026 → menunjuk node sebelumnya

Constructor digunakan untuk menginisialisasi data node ketika objek dibuat

2.3 Operasi Insert Pada DLL

Program InsertDLL_2511532026 digunakan untuk menambahkan node pada Doubly Linked List.

Kode Program:

```
1 package pekan6_2511532026;
2
3 public class InsertDLL_2511532026 {
4
5     static NodeDLL_2511532026 insertBegin_2026 (NodeDLL_2511532026 head, int data) {
6         //buat node baru
7         NodeDLL_2511532026 newNode_2026 = new NodeDLL_2511532026(data);
8         //jadikan pointer prev nya head
9         newNode_2026.next_2026 = head;
10        //jadikan pointer prev head ke new node
11        if (head!=null) {
12            head.prev_2026 = newNode_2026;
13        }
14        return newNode_2026;
15    }
16
17    public static NodeDLL_2511532026 insertEnd_2026(NodeDLL_2511532026 head,int newData) {
18        //buat node baru
19        NodeDLL_2511532026 newNode_2026 = new NodeDLL_2511532026 (newData);
20        //jika dll null jadikan head
21        if (head == null) {
22            head = newNode_2026;
23        }
24        else {
25            NodeDLL_2511532026 curr_2026 = head;
26            while (curr_2026.next_2026 != null) {
27                curr_2026 = curr_2026.next_2026;
28            }
29            curr_2026.next_2026 = newNode_2026;
30            newNode_2026.prev_2026 = curr_2026;
31        }
32        return head;
33    }
34    public static NodeDLL_2511532026 insertAtPosition_2026(NodeDLL_2511532026 head_2026, int pos_2026, int new_data_2026) {
35
36        // Buat node baru
37        NodeDLL_2511532026 new_node_2026 = new NodeDLL_2511532026(new_data_2026);
38
39        if (pos_2026 == 1) {
40
41            new_node_2026.next_2026 = head_2026;
42
43            if (head_2026 != null) {
44                head_2026.prev_2026 = new_node_2026;
45            }
```

```
47            head_2026 = new_node_2026;
48            return head_2026;
49        }
50
51        NodeDLL_2511532026 curr_2026 = head_2026;
52
53        for (int i_2026 = 1; i_2026 < pos_2026 - 1 && curr_2026 != null; ++i_2026) {
54            curr_2026 = curr_2026.next_2026;
55        }
56
57        if (curr_2026 == null) {
58            System.out.println("Posisi tidak ada");
59            return head_2026;
60        }
61
62        new_node_2026.prev_2026 = curr_2026;
63        new_node_2026.next_2026 = curr_2026.next_2026;
64        curr_2026.next_2026 = new_node_2026;
65
66        if (new_node_2026.next_2026 != null) {
67            new_node_2026.next_2026.prev_2026 = new_node_2026;
68        }
69
70        return head_2026;
71    }
72
73    public static void printList_2026(NodeDLL_2511532026 head_2026) {
74
75        NodeDLL_2511532026 curr_2026 = head_2026;
76
77        while (curr_2026 != null) {
78            System.out.print(curr_2026.data_2026 + " <-> ");
79            curr_2026 = curr_2026.next_2026;
80        }
81
82        System.out.println();
83    }
84    public static void main(String[] args) {
85
86        // membuat dll 2 <-> 3 <-> 5
87        NodeDLL_2511532026 head_2026 = new NodeDLL_2511532026(2);
88
89        head_2026.next_2026 = new NodeDLL_2511532026(3);
90        head_2026.next_2026.prev_2026 = head_2026;
```

```

92     head_2026.next_2026.next_2026 = new NodeDLL_2511532026(5);
93     head_2026.next_2026.next_2026.prev_2026 = head_2026.next_2026;
94
95     // cetak DLL awal
96     System.out.print("DLL Awal: ");
97     printList_2026(head_2026);
98
99     // tambah 1 di awal
100    head_2026 = insertBegin_2026(head_2026, 1);
101
102    System.out.print(
103        "simpul 1 ditambah di awal: ");
104
105    printList_2026(head_2026);
106
107    // tambah 6 di akhir
108    System.out.print(
109        "simpul 6 ditambah di akhir: ");
110
111    int data_2026 = 6;
112
113    head_2026 = insertEnd_2026(head_2026, data_2026);
114
115    printList_2026(head_2026);
116
117    // menambah node 4 di posisi 4
118    System.out.print("tambah node 4 di posisi 4: ");
119
120    int data2_2026 = 4;
121    int pos_2026 = 4;
122
123    head_2026 = insertAtPosition_2026(head_2026, pos_2026, data2_2026);
124
125    printList_2026(head_2026);
126
127    }
128 }

```

Operasi yang dilakukan meliputi:

1. Insert di awal (insertBegin_2026)
2. Insert di akhir (insertEnd_2026)
3. Insert pada posisi tertentu (insertAtPosition_2026)

Pada proses insert, pointer next dan prev harus diperbarui agar hubungan antar node tetap benar.

2.4 Operasi Delete Pada DLL

Program HapusDLL_2511532026 digunakan untuk menghapus node pada Doubly Linked List.

Kode Program:

```
1 package pekan6_2511532026;
2 public class HapusDLL_2511532026 {
3
4     // fungsi menghapus node awal
5     public static NodeDLL_2511532026 delHead_2026(NodeDLL_2511532026 head_2026) {
6
7         if (head_2026 == null) {
8             return null;
9         }
10
11         head_2026 = head_2026.next_2026;
12
13         if (head_2026 != null) {
14             head_2026.prev_2026 = null;
15         }
16
17         return head_2026;
18     }
19
20     // fungsi menghapus di akhir
21     public static NodeDLL_2511532026 delLast_2026(NodeDLL_2511532026 head_2026) {
22
23         if (head_2026 == null) {
24             return null;
25         }
26
27         if (head_2026.next_2026 == null) {
28             return null;
29         }
30
31         NodeDLL_2511532026 curr_2026 = head_2026;
32
33         while (curr_2026.next_2026 != null) {
34             curr_2026 = curr_2026.next_2026;
35         }
36
37         // update pointer previous node
38         if (curr_2026.prev_2026 != null) {
39             curr_2026.prev_2026.next_2026 = null;
40         }
41
42         return head_2026;
43     }
44
45     // fungsi menghapus node posisi tertentu
46     public static NodeDLL_2511532026 delPos_2026(NodeDLL_2511532026 head_2026, int pos_2026) {
47
48         // jika DLL kosong
49         if (head_2026 == null) {
50             return head_2026;
51         }
52
53         NodeDLL_2511532026 curr_2026 = head_2026;
54
55         // telusuri sampai ke node yang akan dihapus
56         for (int i_2026 = 1; curr_2026 != null && i_2026 < pos_2026; ++i_2026) {
57             curr_2026 = curr_2026.next_2026;
58         }
59
60         // jika posisi tidak ditemukan
61         if (curr_2026 == null) {
62             return head_2026;
63         }
64
65         // update pointer
66         if (curr_2026.prev_2026 != null) {
67             curr_2026.prev_2026.next_2026 = curr_2026.next_2026;
68         }
69
70         if (curr_2026.next_2026 != null) {
71             curr_2026.next_2026.prev_2026 = curr_2026.prev_2026;
72         }
73
74         // jika yang dihapus head
75         if (head_2026 == curr_2026) {
76             head_2026 = curr_2026.next_2026;
77         }
78
79         return head_2026;
80     }
81
82     // fungsi mencetak DLL
83     public static void printList_2026(NodeDLL_2511532026 head_2026) {
84
85         NodeDLL_2511532026 curr_2026 = head_2026;
86
87         while (curr_2026 != null) {
88             System.out.print(curr_2026.data_2026 + " ");
89             curr_2026 = curr_2026.next_2026;
90         }
91     }
92 }
```

```

91     System.out.println();
92 }
93 public static void main(String[] args) {
94     //buat sebuah dll
95     NodeDLL_2511532026 head_2026 = new NodeDLL_2511532026(1);
96     head_2026.next_2026 = new NodeDLL_2511532026(2);
97     head_2026.next_2026.prev_2026 = head_2026;
98     head_2026.next_2026.next_2026 = new NodeDLL_2511532026(3);
99     head_2026.next_2026.next_2026.prev_2026 = head_2026.next_2026;
100    head_2026.next_2026.next_2026.next_2026 = new NodeDLL_2511532026(4);
101    head_2026.next_2026.next_2026.next_2026.prev_2026 = head_2026.next_2026.next_2026;
102    head_2026.next_2026.next_2026.next_2026.next_2026 = new NodeDLL_2511532026 (5);
103    head_2026.next_2026.next_2026.next_2026.next_2026.prev_2026 = head_2026.next_2026.next_2026;
104    System.out.print("DLL Awal : ");
105    printList_2026(head_2026);
106
107    System.out.print("Setelah head di hapus : ");
108    head_2026 = delHead_2026(head_2026);
109    printList_2026(head_2026);
110
111    System.out.print("Setelah node terakhir di hapus : ");
112    head_2026 = delLast_2026(head_2026);
113    printList_2026(head_2026);
114
115    System.out.print("menghapus node ke-2 : ");
116    head_2026 = delPos_2026(head_2026, 2);
117
118    printList_2026(head_2026);
119 }
120
121 }

```

Operasi yang dilakukan yaitu:

1. Menghapus node awal (delHead_2026)
2. Menghapus node akhir (delLast_2026)
3. Menghapus node pada posisi tertentu (delPos_2026)

Saat node dihapus, pointer node lain harus diperbarui agar list tidak terputus.

2.5 Operasi Traversal Pada DLL

Program PenelusuranDLL_2511532026 digunakan untuk melakukan penelusuran data.

Kode Program:

```
1 package pekan6_2511532026;
2 public class PenelusuranDLL_2511532026 {
3
4     // fungsi penelusuran maju
5     static void forwardTraversal_2026(NodeDLL_2511532026 head_2026) {
6
7         // memulai penelusuran dari head
8         NodeDLL_2511532026 curr_2026 = head_2026;
9
10        // lanjutkan sampai akhir
11        while (curr_2026 != null) {
12
13            // print data
14            System.out.print(curr_2026.data_2026 + " <-> ");
15
16            // pindah ke node berikutnya
17            curr_2026 = curr_2026.next_2026;
18        }
19
20        // print spasi
21        System.out.println();
22    }
23
24    // fungsi penelusuran mundur
25    static void backwardTraversal_2026(NodeDLL_2511532026 tail_2026) {
26
27        // mulai dari akhir
28        NodeDLL_2511532026 curr_2026 = tail_2026;
29
30        // lanjut sampai head
31        while (curr_2026 != null) {
32
33            // cetak data
34            System.out.print(curr_2026.data_2026 + " <-> ");
35
36            // pindah ke node sebelumnya
37            curr_2026 = curr_2026.prev_2026;
38        }
39
40        // cetak spasi
41        System.out.println();
42    }
43
44    public void main(String[] args) {
45        NodeDLL_2511532026 head_2026 = new NodeDLL_2511532026(1);
46        NodeDLL_2511532026 second_2026 = new NodeDLL_2511532026(2);
47
48        head_2026.next_2026 = second_2026;
49        second_2026.prev_2026 = head_2026;
50        second_2026.next_2026 = third_2026;
51        third_2026.prev_2026 = second_2026;
52
53        System.out.println ("Penelusuran maju:");
54        forwardTraversal_2026 (head_2026);
55        System.out.println ("Penelusuran mundur:");
56        backwardTraversal_2026 (third_2026);
57
58    }
59
60 }
```

Terdapat dua jenis traversal:

1. Forward Traversal → penelusuran dari depan ke belakang
2. Backward Traversal → penelusuran dari belakang ke depan

Traversal dilakukan menggunakan perulangan hingga node bernilai null.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Doubly Linked List merupakan struktur data yang memungkinkan penelusuran dua arah menggunakan pointer next dan prev. Implementasi Doubly Linked List dalam Java dapat dilakukan dengan membuat class node dan operasi dasar seperti insert, delete, serta traversal. Program yang dibuat berhasil menunjukkan cara kerja penambahan, penghapusan, dan penelusuran node pada Doubly Linked List dengan baik.

TUGAS DLL

Program Lagu_2511532026

Kode Program:

```
1 package pekan6_2511532026;
2 public class Lagu_2511532026 {
3
4     String judul_2026;
5     String penyanyi_2026;
6
7     Lagu_2511532026 next_2026;
8     Lagu_2511532026 prev_2026;
9
10    // Constructor
11    public Lagu_2511532026(String judul_2026, String penyanyi_2026) {
12
13        this.judul_2026 = judul_2026;
14        this.penyanyi_2026 = penyanyi_2026;
15
16        this.next_2026 = null;
17        this.prev_2026 = null;
18    }
19
20    // Getter
21    public String getJudul_2026() {
22        return judul_2026;
23    }
24
25    public String getPenyanyi_2026() {
26        return penyanyi_2026;
27    }
28
29    // Setter
30    public void setJudul_2026(String judul_2026) {
31        this.judul_2026 = judul_2026;
32    }
33
34    public void setPenyanyi_2026(String penyanyi_2026) {
35        this.penyanyi_2026 = penyanyi_2026;
36    }
37 }
38
```

Class Lagu_2511532026 berfungsi sebagai node pada struktur data Double Linked List. Class ini digunakan untuk menyimpan data lagu berupa judul dan penyanyi. Selain itu, class ini juga memiliki dua pointer yaitu next_2026 dan prev_2026. Pointer next_2026 digunakan untuk menunjuk ke node lagu berikutnya, sedangkan prev_2026 digunakan untuk menunjuk ke node lagu sebelumnya. Dengan adanya dua pointer tersebut, setiap node dapat terhubung ke dua arah.

Pada class ini juga terdapat constructor yang digunakan untuk mengisi data lagu ketika object dibuat. Selain constructor, terdapat method getter dan setter yang berfungsi untuk mengambil dan mengubah data judul maupun penyanyi lagu. Class ini menjadi dasar utama dalam pembentukan playlist pada program Double Linked List.

Program Musik_2511532026

Kode Program:

```
1 package pekan6_2511532026;
2 import java.util.Scanner;
3
4 public class Musik_2511532026 {
5
6     Lagu_2511532026 head_2026, tail_2026;
7
8     public void tambahLagu_2026(String judul_2026, String penyanyi_2026) {
9         Lagu_2511532026 baru_2026 = new Lagu_2511532026(judul_2026, penyanyi_2026);
10
11         if (head_2026 == null)
12             head_2026 = tail_2026 = baru_2026;
13     }
14     else {
15         tail_2026.next_2026 = baru_2026;
16         baru_2026.prev_2026 = tail_2026;
17         tail_2026 = baru_2026;
18     }
19     System.out.println("Lagu berhasil ditambahkan!");
20 }
21
22 public void hapusLaguAwal_2026() {
23     if (head_2026 == null) {
24         System.out.println("Playlist kosong!");
25         return;
26     }
27
28     System.out.println("Lagu \"\" + head_2026.judul_2026 + "\" dihapus.");
29
30     if (head_2026 == tail_2026)
31         head_2026 = tail_2026 = null;
32     else {
33         head_2026 = head_2026.next_2026;
34         head_2026.prev_2026 = null;
35     }
36 }
37
38 public void tampilMaju_2026() {
39     if (head_2026 == null) {
40         System.out.println("Playlist kosong!");
41         return;
42     }
43
44     System.out.println("\n=== Playlist Maju ===");
45     for (Lagu_2511532026 temp_2026 = head_2026; temp_2026 != null; temp_2026 = temp_2026.next_2026)
46
47     public void tampilMundur_2026() {
48         if (tail_2026 == null) {
49             System.out.println("Playlist kosong!");
50             return;
51         }
52
53         System.out.println("\n=== Playlist Mundur ===");
54         for (Lagu_2511532026 temp_2026 = tail_2026; temp_2026 != null; temp_2026 = temp_2026.prev_2026)
55             System.out.println(temp_2026.judul_2026 + " - " + temp_2026.penyanyi_2026);
56     }
57
58     public void cariLagu_2026(String cari_2026) {
59         for (Lagu_2511532026 temp_2026 = head_2026; temp_2026 != null; temp_2026 = temp_2026.next_2026) {
60             if (temp_2026.judul_2026.equalsIgnoreCase(cari_2026)) {
61                 System.out.println("Lagu ditemukan!");
62                 System.out.println(temp_2026.judul_2026 + " - " + temp_2026.penyanyi_2026);
63                 return;
64             }
65         }
66
67         System.out.println("Lagu tidak ditemukan!");
68     }
69 }
70
71 public static void main(String[] args) {
72     Scanner input_2026 = new Scanner(System.in);
73     Musik_2511532026 playlist_2026 = new Musik_2511532026();
74     int pilih_2026;
75
76     do {
77         System.out.println("\n=== Playlist Musik NIM: 2511532026 ===");
78         System.out.println("1. Tambah Lagu");
79         System.out.println("2. Hapus Lagu Pertama");
80         System.out.println("3. Lihat Playlist (Maju)");
81         System.out.println("4. Lihat Playlist (Mundur)");
82         System.out.println("5. Cari Lagu");
83         System.out.println("6. Keluar");
84         System.out.print("Pilihan: ");
85
86         pilih_2026 = input_2026.nextInt();
87         input_2026.nextLine();
88     }
```

```

90     switch (pilih_2026) {
91     case 1:
92         System.out.print("Judul Lagu: ");
93         String judul_2026 = input_2026.nextLine();
94         System.out.print("Penyanyi: ");
95         String penyanyi_2026 = input_2026.nextLine();
96         playlist_2026.tambahLagu_2026(judul_2026, penyanyi_2026);
97         break;
98
99         case 2: playlist_2026.hapusLaguAwal_2026(); break;
100        case 3: playlist_2026.tampilMaju_2026(); break;
101        case 4: playlist_2026.tampilMundur_2026(); break;
102
103        case 5:
104            System.out.print("Masukkan judul lagu: ");
105            playlist_2026.cariLagu_2026(input_2026.nextLine());
106            break;
107
108        case 6: System.out.println("Program selesai."); break;
109        default: System.out.println("Pilihan tidak valid!");
110    }
111
112    } while (pilih_2026 != 6);
113
114    input_2026.close();
115 }
116 }

```

Class Musik_2511532026 digunakan untuk mengelola playlist lagu menggunakan struktur data Double Linked List. Class ini memiliki dua pointer utama yaitu head_2026 yang menunjuk ke lagu pertama dan tail_2026 yang menunjuk ke lagu terakhir. Dengan menggunakan head dan tail, program dapat melakukan traversal dari depan maupun dari belakang playlist.

Pada class ini terdapat beberapa method utama. Method tambahLagu_2026() digunakan untuk menambahkan lagu baru di akhir playlist. Method hapusLaguAwal_2026() digunakan untuk menghapus lagu pertama pada playlist. Method tampilMaju_2026() berfungsi menampilkan daftar lagu dari awal ke akhir menggunakan pointer next_2026, sedangkan tampilMundur_2026() digunakan untuk menampilkan playlist dari akhir ke awal menggunakan pointer prev_2026.

Selain itu terdapat method cariLagu_2026() yang digunakan untuk mencari lagu berdasarkan judul secara tidak case-sensitive menggunakan equalsIgnoreCase(). Pada class ini juga terdapat method main yang berfungsi sebagai program utama untuk menampilkan menu, menerima input pengguna, dan menjalankan operasi playlist sesuai pilihan user

Output

Musik_2511532026 [Java Application] C:\Users\ryhiyhu\Downloa

```
=== Playlist Musik NIM: 2511532026 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan:
```

Musik_2511532026 [Java Application] C:\Users\ryhiyhu\Downlo

```
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: Her way
Penyanyi: PartyNextDoor
Lagu berhasil ditambahkan!
```

Musik_2511532026 [Java Application] C:\Users\ryhiyhu

```
=== Playlist Musik NIM: 2511532026 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu "Her way" dihapus.
```